COMPLETE GUIDE

GraphQL

From Zero to Hero, with Express.js

Part I: Theory (concepts, type system, query language, execution, advanced topics)

Part II: Code (a complete, tested Express + Apollo Server blog API with auth, DataLoader, pagination and subscriptions)

Prepared for Subhamoy

October 20, 2026 | Diagrams generated with Graphviz

Contents

Part I: Theory

- 1. What is GraphQL and why does it exist?
- 2. Core ideas and vocabulary
- 3. The type system and SDL
- 4. The query language
- 5. How a request is executed
- 6. Mutations and subscriptions
- 7. Advanced theory (N+1, pagination, auth, caching, security, federation)
- 8. GraphQL vs REST at a glance

Part II: Code with Express.js

- 9. Setup and your first GraphQL server
- 10. The real project: a blog API (architecture, structure, packages)
- 11. Walking through the code (every file explained)
- 12. Trying every feature (queries, auth, DataLoader, subscriptions)
- 13. Testing, errors and production hardening
- 14. Calling your API from a client
- 15. Becoming a hero: practice and next steps
- 16. Cheat sheet and glossary

> **TIP** How to use this guide: read Part I once end to end, then build Part II while keeping Part I open. Every file of the project is shown in full, and all outputs were produced by running the code.

PART I

Theory

Understand GraphQL deeply before writing a single resolver.

1. What is GraphQL and why does it exist?

GraphQL is a **query language for APIs** and a **runtime for executing those queries** against your data. It was created at Facebook in 2012 to power the News Feed in their mobile apps, open-sourced in 2015, and is now governed by the GraphQL Foundation (under the Linux Foundation).

The one-sentence pitch: **the client describes exactly the data it needs, in one request, and the server returns exactly that shape, nothing more and nothing less.**

1.1 The problems with classic REST

REST is great, but as apps (especially mobile apps and complex UIs) grew, three pain points kept appearing:

- **Over-fetching:** `GET /users/1` returns 30 fields, but your header only needs `name` and `avatar`. You waste bandwidth and battery.
- **Under-fetching and multiple round trips:** one screen needs the user, their posts, and the comments on each post. That is 3+ requests (often sequential), each paying network latency.
- **Versioning and endpoint sprawl:** every new screen needs a new endpoint or a new `/v2`. Backend and frontend teams constantly negotiate response shapes.

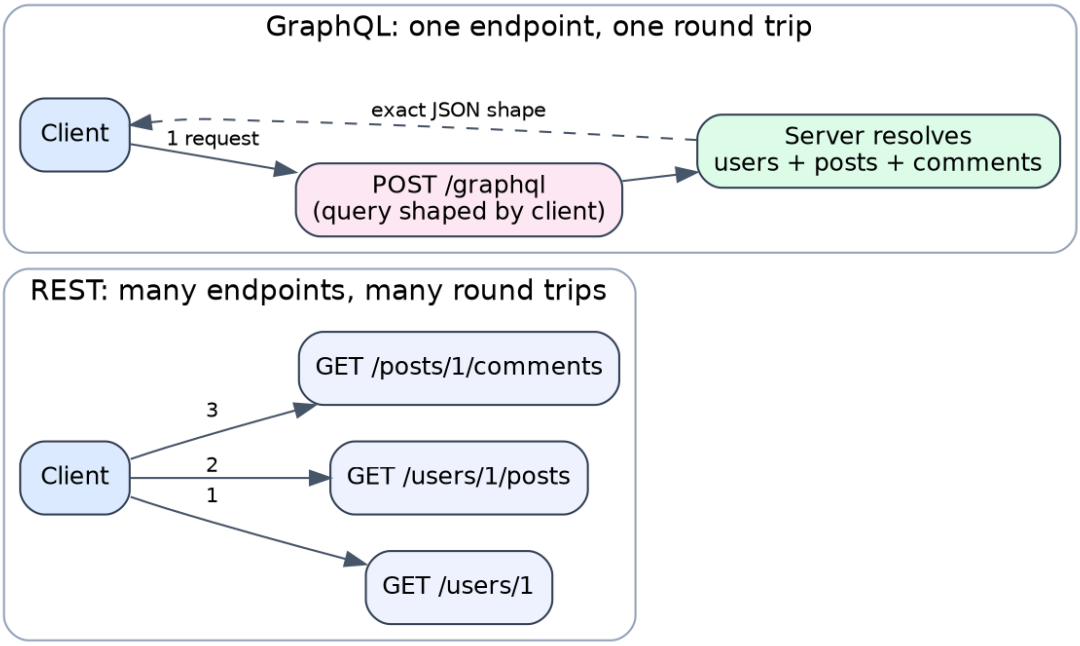


Figure 1.1: REST needs several endpoints; GraphQL uses one endpoint and a client-shaped query.

1.2 A first look

Here is the entire idea in one example. The client sends this query:

```
request (query text)
query {
  user(id: "1") {
    name
```

```
    posts {
      title
      comments { text }
    }
  }
}
```

and the server answers with JSON that mirrors the query exactly:

```
response (JSON)
{
  "data": {
    "user": {
      "name": "Asha",
      "posts": [
        { "title": "GraphQL 101", "comments": [{ "text": "Great post!" }] }
      ]
    }
  }
}
```

> **KEY IDEA** The response has the same shape as the query. If you did not ask for a field, you do not get it. If you ask for it, you always get it (or an error).

1.3 When to use GraphQL (and when not to)

GraphQL shines when...	Think twice when...
Many different clients (web, iOS, Android, partners) need different slices of the same data	You have a tiny CRUD API with one client; REST is simpler
Data is a graph (users, posts, comments, tags) and screens traverse relationships	You mostly upload/download files or stream large binary data
You want to evolve the API without versioning (add fields, deprecate old ones)	You rely heavily on plain HTTP caching/CDNs for public GET endpoints
You aggregate several backends/microservices behind one API (a gateway/BFF)	The team cannot invest in schema design, DataLoader, and security hardening

> **NOTE** GraphQL is **not** a database, not a replacement for your database, and not tied to any language or storage. It is just a contract and an execution model that sits between clients and your data sources.

2. Core ideas and vocabulary

Before diving deep, memorise these building blocks. Everything else in this guide is an elaboration of them.

Term	Meaning
Schema	The contract: every type, field, and operation the API supports. Written in SDL (Schema Definition Language).
Type	A named shape of data, such as `User` or `Post`.
Field	A single piece of data on a type, such as `User.name`. Fields can take arguments.
Operation	One of three kinds: `query` (read), `mutation` (write), `subscription` (stream).
Resolver	A function that knows how to produce the value for one field.
Context	A per-request object shared by all resolvers (current user, DB handles, loaders).
Selection set	The `{ ... }` list of fields the client asks for.
Variables	Typed inputs passed separately from the query text (`$id`).
Introspection	The schema can describe itself; that powers docs, autocomplete, and codegen.

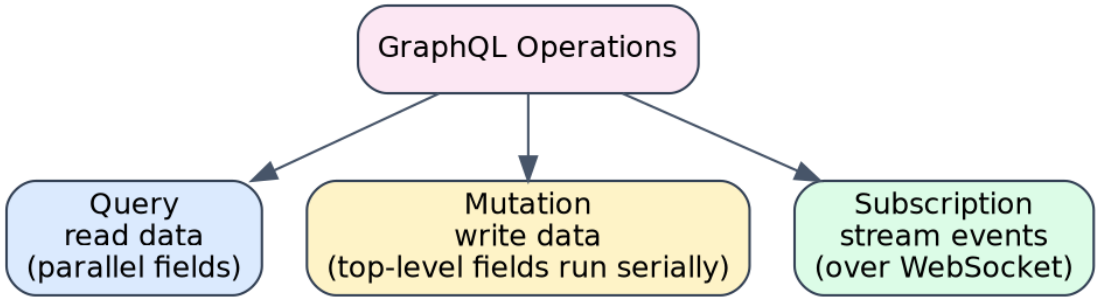


Figure 2.1: The three operation types.

2.1 Key properties of GraphQL

- **Single endpoint.** Usually `POST /graphql`. The query text (not the URL) says what you want.
- **Strongly typed.** Every field has a type. The server validates queries against the schema before running any code.
- **Hierarchical.** Queries are trees that follow relationships in your data graph.
- **Client-specified.** The client controls the response shape; the server controls what is possible.
- **Introspective.** Tools like Apollo Sandbox, GraphiQL, and code generators read your schema automatically.
- **Transport-agnostic.** In practice it runs over HTTP (queries/mutations) and WebSocket or SSE (subscriptions), but the spec does not require it.

2.2 Roadmap of this guide

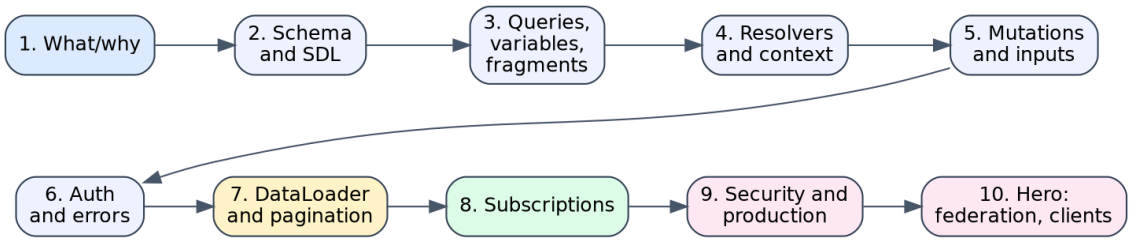


Figure 2.2: The learning path from zero to hero.

3. The type system and SDL

The schema is the heart of GraphQL. You write it in **SDL** (Schema Definition Language), which looks like a simplified, language-neutral version of TypeScript interfaces.

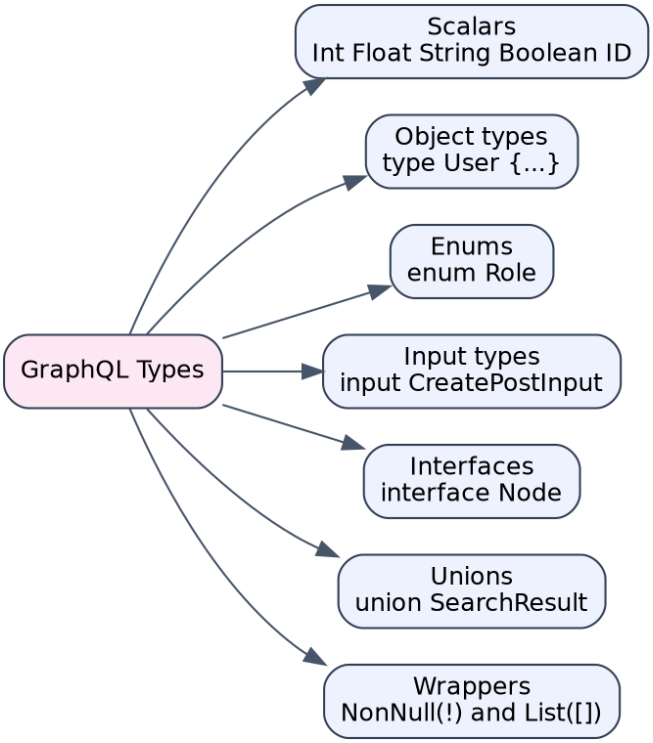


Figure 3.1: The families of types in GraphQL.

3.1 Scalars

Scalars are the leaves of every query, the actual values. The built-in scalars are:

Scalar	Description	Example
`Int`	Signed 32-bit integer	`42`
`Float`	Double-precision float	`3.14`
`String`	UTF-8 text	`"hello"`
`Boolean`	true or false	`true`
`ID`	Unique identifier, serialised as a string (often not human-readable)	`"abc123"`

You can define **custom scalars** such as `DateTime`, `JSON`, `Email`, or `URL` (libraries like `graphql-scalars` provide many).

3.2 Object types and fields

SDL

```
type User {
```

```
  id: ID!
  name: String!
  email: String!
  posts: [Post!]!
}
```

3.3 Nullability and lists (the "!" and "[]")

In GraphQL every type is **nullable by default**. Add `!` to make it non-null. Lists wrap a type in brackets, and nullability applies at **both** levels:

Type	Meaning
`String`	A string or `null`
`String!`	Always a string, never `null`
`[String]`	A list that may be `null`, whose items may be `null`
`[String!]`	A list that may be `null`, but items are never `null`
`[String]!`	Never-null list, items may be `null`
`[String!]!`	Never-null list, never-null items (most common for "has many")

> PITFALL **Null bubbling:** if a non-null field fails (throws or returns null), GraphQL cannot return `null` for it, so it sets the nearest **nullable parent** to `null` instead. Overusing `!` can make one failing field wipe out a large part of the response. Use `!` deliberately.

3.4 Arguments

Any field can take arguments, with optional default values:

SDL
```
type Query {
  posts(first: Int = 10, after: String): PostConnection!
  user(id: ID!): User
}
```

3.5 Enums

SDL
```
enum Role {
  USER
  ADMIN
}
```

Enums restrict a field to a fixed set of values, and tools can autocomplete them.

3.6 Input types

Arguments with many fields are cleaner as **input types**. Input types are for **arguments only**; they cannot have resolvers or reference output object types.

SDL
```
input CreatePostInput {
  title: String!
  body: String!
  published: Boolean = false
}

type Mutation {
  createPost(input: CreatePostInput!): Post!
}
```

3.7 Interfaces

An interface is a list of fields that several types must implement. Querying the interface gives you only the shared fields; use inline fragments for type-specific ones.

SDL
```
interface Node {
  id: ID!
}

type User implements Node { id: ID!  name: String! }
type Post implements Node { id: ID!  title: String! }

type Query {
  node(id: ID!): Node
}
```

3.8 Unions

A union says "this field is one of these types" with no shared fields required. It is great for search results and error-or-result patterns.

SDL
```
union SearchResult = User | Post | Comment

type Query {
  search(term: String!): [SearchResult!]!
}
```

3.9 Descriptions and directives

SDL
```
"""
A person who can write posts.
"""
type User {
  "Display name shown in the UI"
  name: String!
  nickname: String @deprecated(reason: "Use name instead")
}
```

Built-in directives: `@deprecated(reason)` marks a schema field as deprecated, while `@skip(if:)` and `@include(if:)` conditionally drop or keep fields **in a query** (see chapter 4). Servers can define **custom directives** (for example `@auth`, `@cacheControl`).

3.10 The three root types

Every schema has a `Query` root type; it may also have `Mutation` and `Subscription`. These are the entry points: every operation starts at one of them.

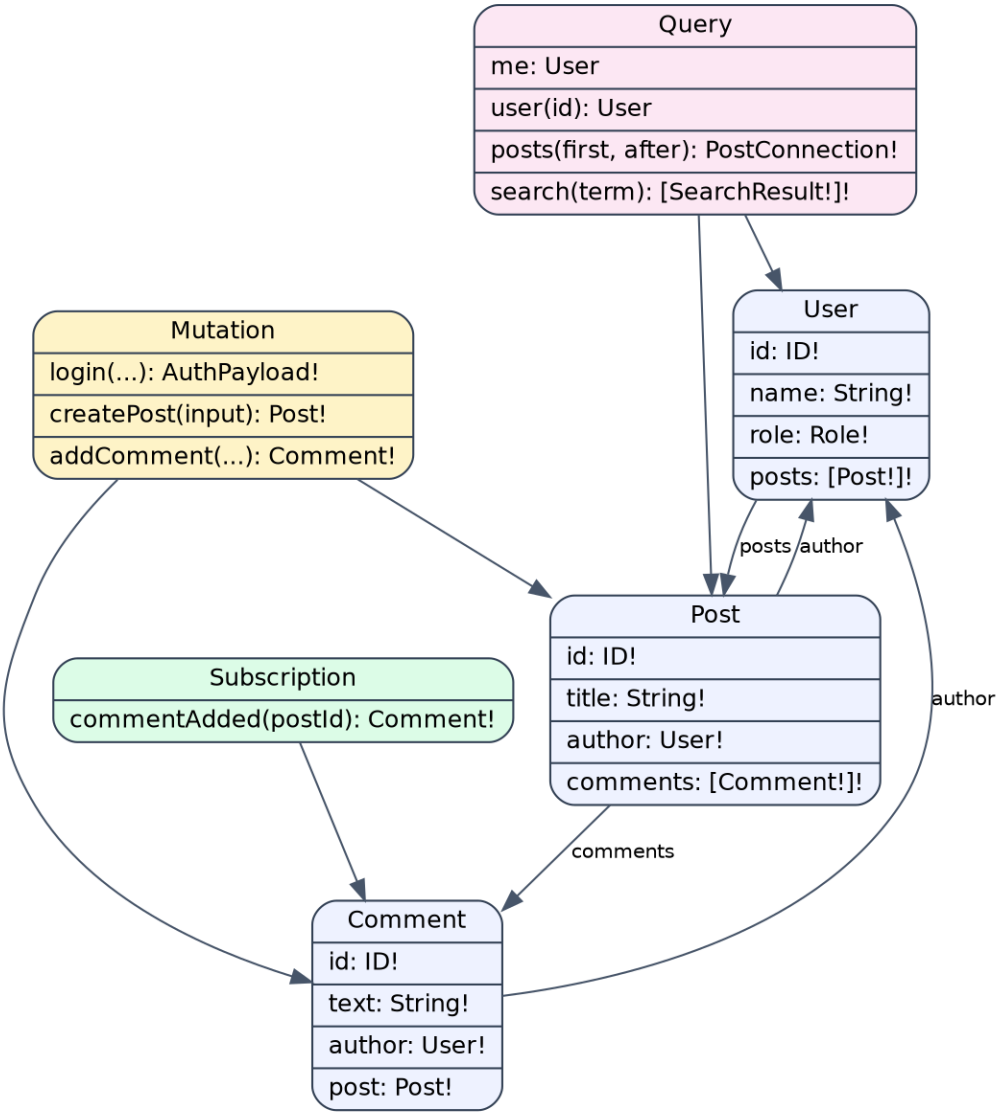


Figure 3.2: Our blog schema as a graph. Root types (top) lead into a connected graph of object types.

> **TIP** Design the schema around **how clients think about the domain**, not around your database tables. `Post.author` is a field returning a `User`, even if the database stores `authorId`.

4. The query language

4.1 Fields and selection sets

query

```
{
  me {
    id
    name
  }
}
```

The shorthand `{ ... }` is an anonymous query. Every selection must end in scalar or enum leaves, so you cannot just write `{ me }` when `me` is an object.

4.2 Arguments

query

```
{
  user(id: "2") { name }
  posts(first: 5) { totalCount }
}
```

4.3 Aliases

Aliases rename the result key, which lets you fetch the same field with different arguments in one request:

query

```
{
  firstPost: post(id: "1") { title }
  secondPost: post(id: "2") { title }
}
```

response

```
{ "data": { "firstPost": { "title": "GraphQL 101" }, "secondPost": { "title": "Express Tips" } } }
```

4.4 Operation names and variables

In real apps never build query strings by concatenation. Declare **variables** with types and send their values separately. This is safer, cacheable, and lets tooling validate them.

query

```
query GetPost($id: ID!, $withComments: Boolean = false) {
  post(id: $id) {
    title
    comments @include(if: $withComments) { text }
  }
}
```

HTTP POST body (JSON)

```
{
  "query": "query GetPost($id: ID!) { post(id: $id) { title } }",
  "variables": { "id": "1" },
  "operationName": "GetPost"
}
```

The three fields in the body are standard: `query`, `variables`, and `operationName` (needed when the document contains several operations).

4.5 Fragments

A **fragment** is a reusable set of fields on a type. They keep queries DRY and let UI components declare their own data needs (the idea behind Relay and Apollo Client colocation).

query with a fragment

```
fragment PostPreview on Post {
  id
  title
  author { name }
}

{
  posts(first: 5) {
    edges { node { ...PostPreview } }
  }
}
```

4.6 Inline fragments and __typename

When a field returns an interface or union, use `... on Type` to pick type-specific fields. `__typename` is a special field available everywhere that tells you the concrete type name.

query

```
{
  search(term: "great") {
    __typename
    ... on User    { name }
    ... on Post    { title }
    ... on Comment { text }
  }
}
```

4.7 Directives in queries

query

```
query($showEmail: Boolean!) {
  me {
    name
    email @include(if: $showEmail)
    role  @skip(if: $showEmail)
  }
}
```

4.8 Mutations

Mutations look like queries but use the `mutation` keyword, and by convention they **change data and return the changed object**, so the client can update its UI without a second request.

mutation

```
mutation CreatePost($input: CreatePostInput!) {
  createPost(input: $input) {
    id
    title
    author { name }
  }
}
```

4.9 Introspection

You can ask a server about its own schema. This is how GraphiQL, Apollo Sandbox, and codegen tools work.

```
introspection query
{
  __schema { types { name kind } }
  __type(name: "Post") { fields { name type { name kind } } }
}
```

5. How a request is executed

5.1 The lifecycle

Every GraphQL request goes through the same phases. Understanding them tells you exactly where errors come from and where to hook in logic.

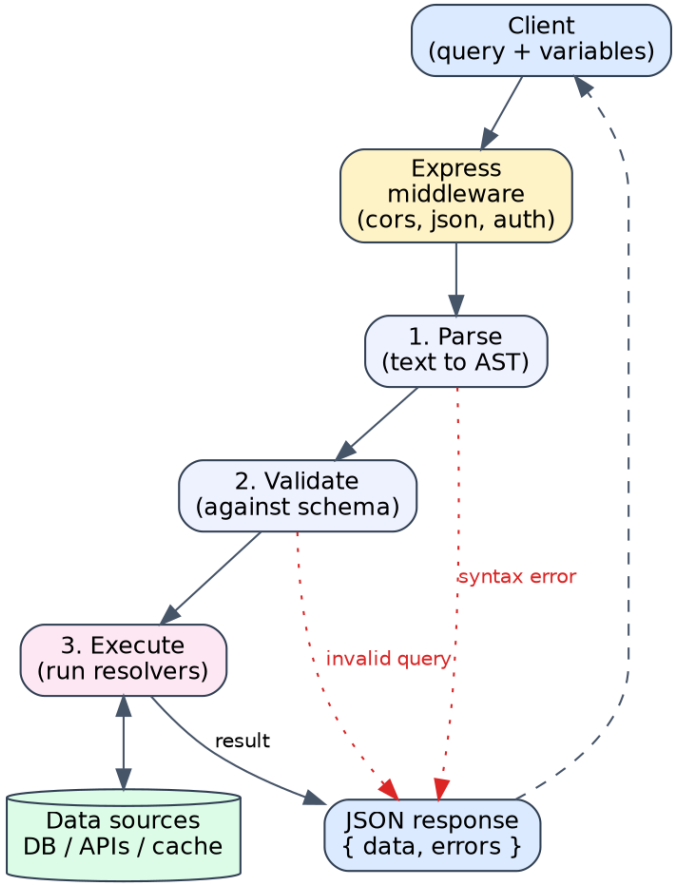


Figure 5.1: Request lifecycle: parse, validate, execute.

1. **Parse:** the query text becomes an AST (abstract syntax tree). A typo like a missing brace fails here (`GRAPHQL_PARSE_FAILED`).
2. **Validate:** the AST is checked against the schema: do fields exist, are argument types right, are fragments valid? Failure gives `GRAPHQL_VALIDATION_FAILED`, and **no resolver runs**. You can add custom rules here (depth limits, cost limits).
3. **Execute:** the engine walks the query tree, calling a resolver for each field, and assembles the result.
4. **Respond:** the result is `{ data, errors?, extensions? }`.

5.2 Resolvers

A resolver is a function with four arguments:

```
resolver signature
function resolver(parent, args, context, info) {
```

```
  // parent  : the result of the parent field's resolver (e.g. the Post when resolving Post.author)
  // args    : the arguments passed to this field (e.g. { id: "1" })
  // context : the shared per-request object (viewer, loaders, db)
  // info    : AST + schema details (field name, path), rarely needed
}
```

If you do not supply a resolver for a field, GraphQL uses the **default resolver**: it returns `parent[fieldName]` (calling it if it is a function). That is why a plain object with `title` just works.

resolver map

```
const resolvers = {
  Query: {
    post: (_, { id }, ctx) => ctx.loaders.postById.load(id),
  },
  Post: {
    // 'title' has no resolver: default resolver returns post.title
    author: (post, _, ctx) => ctx.loaders.userById.load(post.authorId),
  },
};
```

5.3 The execution tree

Execution is **depth-first through the tree, level by level**. The resolver for a child field receives the resolved parent object as its first argument. List fields call the child resolvers once per item.

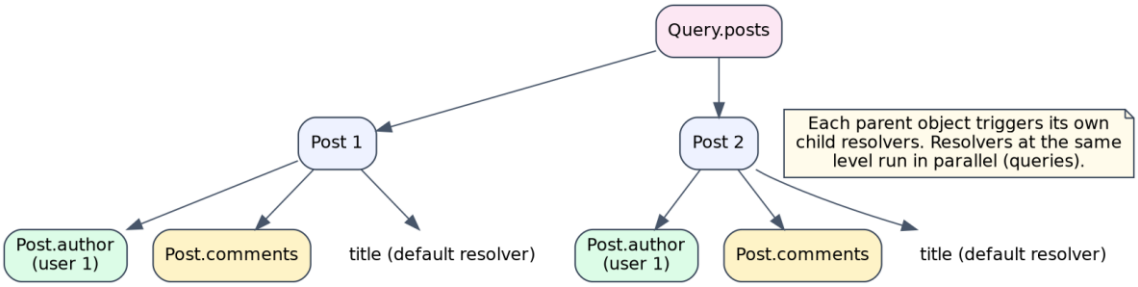


Figure 5.2: Each parent triggers resolvers for its child fields.

- Sibling fields of a **query** may run **in parallel** (when resolvers are async).
- Top-level fields of a **mutation** run **one after another, in order**, so `createUser` then `createPost` is safe.
- A resolver may return a value, a Promise, or an async generator (for subscriptions).

5.4 Context

Context is created **once per request** and handed to every resolver. It is where authentication results, database clients, and DataLoaders live.

context factory (Express integration)

```
context: async ({ req }) => ({
  viewer: getViewer(req.headers.authorization), // who is calling?
  loaders: createLoaders(),                     // fresh caches for this request
})
```

5.5 Errors and partial data

GraphQL can return **data and errors together**. If one field fails, the rest of the query still succeeds, and the failed field becomes `null` with an entry in `errors` containing a `path`.

```
partial success
{
  "data": { "post": { "title": "GraphQL 101", "author": null } },
  "errors": [{
    "message": "Author service unavailable",
    "path": ["post", "author"],
    "extensions": { "code": "INTERNAL_SERVER_ERROR" }
  }]
}
```

Kind of failure	Where	Typical HTTP status (Apollo Server)
Syntax error in query text	Parse phase	400
Unknown field / wrong argument type	Validation phase	400
Exception inside a resolver	Execution phase	200 with `errors` array
Not logged in (we choose 401 via `extensions.http`)	Resolver	401 (customised in our code)
Malformed JSON / CSRF protection	HTTP layer	400

> **NOTE** HTTP 200 does not mean "no errors". Clients must always check the `errors` field. We measured the status codes above against the project built in Part II.

Put machine-readable codes in `extensions.code` (for example `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `BAD_USER_INPUT`) so clients can react without parsing messages.

6. Mutations and subscriptions

6.1 Mutation design rules

- Name mutations as verbs: `createPost`, `updatePost`, `deletePost`, `addComment`.
- Take a single **input object** (`input: CreatePostInput!`), which makes evolving the API easy.
- Return the **affected object** (or a payload type) so clients can update caches.
- For partial updates use an input with all-optional fields (`UpdatePostInput`).
- For expected business failures (such as "email taken"), consider returning a **payload with a union or errors field** instead of throwing, so clients can handle it in a typed way.

SDL: typed errors via union

```
union CreatePostResult = Post | ValidationError

type ValidationError { field: String!  message: String! }

type Mutation {
  createPost(input: CreatePostInput!): CreatePostResult!
}
```

6.2 Subscriptions: real-time data

A subscription is a long-lived operation. The client subscribes once and the server **pushes** a result every time a matching event happens. Queries and mutations use plain HTTP request/response, so subscriptions usually run over **WebSocket** (the `graphql-ws` protocol) or **Server-Sent Events**.

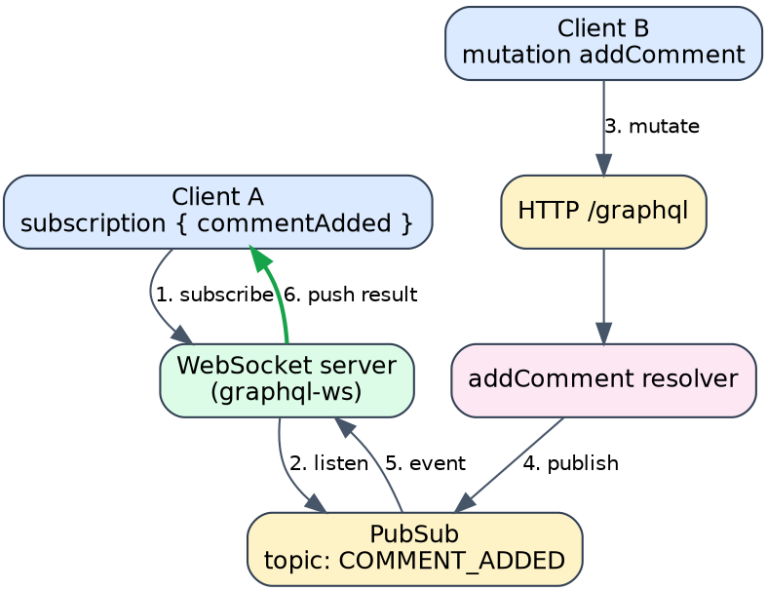


Figure 6.1: Mutation publishes an event; subscribed clients receive it over WebSocket.

- A **PubSub** (publish/subscribe) component connects the mutation to the subscription. In-memory `PubSub` works for one server process; with several servers use Redis, Kafka, or similar.
- The subscription resolver returns an **async iterator**; each yielded value is executed against the selection set the client asked for.

- Use subscriptions for chats, live dashboards, and notifications. For rarely-changing data, **polling** a query is often enough and simpler.

7. Advanced theory (the road to hero)

7.1 The N+1 problem and DataLoader

Resolvers run **per object**. Fetch 100 posts, then resolve `Post.author` for each, and a naive implementation runs **1 query for posts + 100 queries for authors**, even if there are only 5 distinct authors. This is the famous **N+1 problem**.

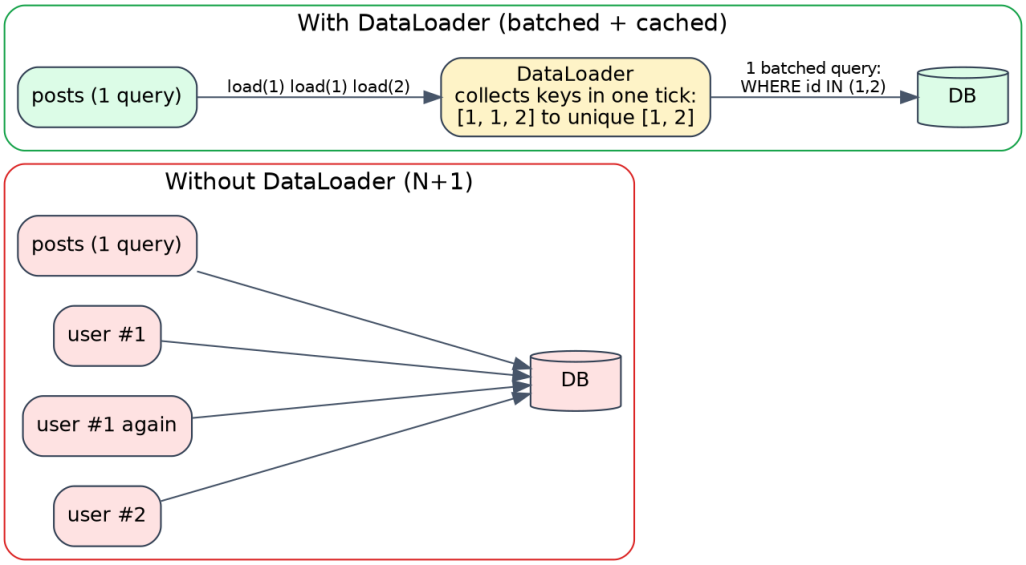


Figure 7.1: DataLoader collects keys within one tick of the event loop and fetches them in a single batch.

DataLoader (a tiny library from Facebook) fixes this with two tricks:

- **Batching:** all `load(key)` calls made in the same tick are collected, then your batch function runs **once** with all the keys.
- **Per-request caching:** the same key loaded twice in one request is fetched once.

DataLoader pattern

```
const userById = new DataLoader(async (ids) => {
  const rows = await db.users.findMany({ where: { id: { in: ids } } }); // ONE query
  const map = new Map(rows.map((u) => [u.id, u]));
  return ids.map((id) => map.get(id) ?? null); // MUST match the order and length of ids
});

// in a resolver:
author: (post, _, ctx) => ctx.loaders.userById.load(post.authorId)
```

> **PITFALL** **Two rules you must never break:**
>
> 1) The batch function must return an array **the same length and order** as the keys. 2) Create loaders **per request** (inside `context`), never as a global, otherwise one user's cached data can leak into another user's response.

7.2 Pagination

Style	How	Pros	Cons
Offset	`posts(limit: 10, offset: 20)`	Simple, jump to any page	Slow on big tables; duplicates/skips when data changes
Cursor (Relay connections)	`posts(first: 10, after: "cursor")`	Stable, efficient, great for infinite scroll	No random page jumps; more schema boilerplate

The **connection pattern** returns `edges { cursor node }`, `pageInfo { hasNextPage endCursor }`, and optionally `totalCount`. We implement it in Part II.

7.3 Authentication vs authorization

- **Authentication (who are you?)** happens **once per request**, usually in the HTTP layer or the `context` function: verify the JWT or session and put `viewer` in context.
- **Authorization (what may you do?)** belongs in the **business layer or resolvers**, close to the data: "only the author or an admin may edit this post".
- Do **not** rely on hiding fields in the client. Check permissions on the server for every field and mutation that needs them.
- Prefer reusable helpers (`requireAuth`, `requireRole`) or directives over copy-pasted checks.

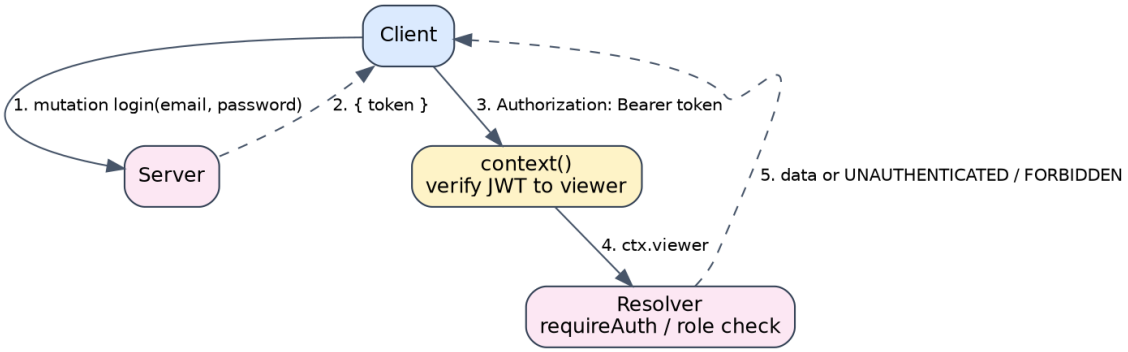


Figure 7.2: Login returns a token; context turns it into a viewer; resolvers authorize.

7.4 Caching

- **Client normalised cache** (Apollo Client, urql, Relay): stores objects by `__typename + id`, so updating one `Post` updates every screen showing it. This is why returning ids and the changed object from mutations matters.
- **HTTP caching** is harder because queries are usually POST requests to one URL. Solutions: **persisted queries** (send a hash instead of the text, optionally via GET so CDNs can cache) and cache hints (`@cacheControl`).
- **Server-side caching:** per-request DataLoader cache, plus Redis or in-memory caches around expensive resolvers or data sources.

7.5 Security checklist

Risk	Mitigation
Very deep queries (`author { posts { author { posts ... } } }`)	**Depth limit** validation rule (we use `graphql-depth-limit`)
Expensive wide queries	**Query cost / complexity analysis**, pagination limits (cap `first`), timeouts
Abusive clients	Rate limiting per IP/user; allow-list **persisted queries** in production
Schema exposure	Disable introspection in production if the API is private (not a substitute for real auth)
Cross-site request forgery	Apollo Server enables CSRF prevention by default (rejects simple `text/plain` POSTs)
Leaking internals	Hide stack traces and mask unexpected error messages in production
Broken access control	Authorize in resolvers/business layer for every sensitive field; test it
Injection	Use variables and parameterised DB queries; never concatenate user input into SQL or queries

7.6 Schema design best practices

- Design for the **client use cases**, not DB tables. Think in a graph.
- Prefer **evolution over versioning**: add fields freely, mark old ones `@deprecated`, remove them after clients have migrated.
- Use **specific mutation names and input objects**. Avoid generic `updateEverything`.
- Make fields non-null only when you can truly guarantee them.
- Use enums for fixed sets, custom scalars for dates, and `ID` for identifiers.
- Use consistent naming: `camelCase` fields, `PascalCase` types, `SCREAMING_SNAKE_CASE` enum values.
- Document with `"""description"""` strings, since they appear in tooling.

7.7 Schema-first vs code-first

Approach	What you write	Tools
Schema-first (SDL-first)	`.graphql`/SDL strings, then resolvers that match. Our project uses this.	graphql-js, Apollo Server, graphql-tools, GraphQL Yoga
Code-first	TypeScript/JS code that defines types; SDL is generated.	Pothos, Nexus, TypeGraphQL, NestJS code-first

7.8 Federation and scaling out

As organisations grow, one GraphQL server becomes a bottleneck for teams. **Federation** (Apollo Federation, also supported by other routers) lets each team own a **subgraph** and a **router/gateway** composes them into one **supergraph** that clients see as a single API. Alternatives include schema stitching and GraphQL Mesh. You do not need this on day one; just know it exists.

7.9 The ecosystem map

Need	Popular choices
Reference implementation	`graphql` (graphql-js)
Spec-compliant HTTP handler	`graphql-http` (what we use in the hello-world)
Full-featured server	Apollo Server, GraphQL Yoga, Mercurius (Fastify)
WebSocket subscriptions	`graphql-ws`
Clients	Apollo Client, urql, Relay, `graphql-request`
Code generation (types)	GraphQL Code Generator
Code-first schema builders	Pothos, Nexus, TypeGraphQL
Batching/caching	DataLoader
Tooling	Apollo Sandbox, GraphiQL, Postman, Insomnia

8. GraphQL vs REST at a glance

Topic	REST	GraphQL
Endpoints	Many (`/users`, `/posts/1`)	One (`/graphql`)
Response shape	Decided by the server	Decided by the client's query
Over/under-fetching	Common	Avoided by design
Round trips for related data	Several	Usually one
Versioning	`/v1`, `/v2`	Evolve the schema, deprecate fields
Type system and docs	Optional (OpenAPI)	Built in, always in sync, introspectable
HTTP caching	Easy (GET + URL)	Harder (needs persisted queries or client caches)
Error handling	HTTP status codes	`errors` array, partial results, status often 200
File upload	Natural (multipart)	Needs extra spec/library
Real time	WebSocket/SSE separately	Subscriptions in the same schema
Learning curve	Low	Medium (schema design, N+1, security)

> **TIP** REST and GraphQL are not enemies. Many companies keep REST for simple public endpoints and add a GraphQL layer for rich clients, often **wrapping existing REST services as data sources**.

PART II

Code with Express.js

Build a complete, tested GraphQL API: auth, relations, DataLoader, pagination and live subscriptions.

9. Setup and your first GraphQL server

Everything in this part was **actually run and tested** (Node.js 22, Express 5, Apollo Server 5, graphql 16). The outputs shown are real.

9.1 Requirements

- Node.js 20 or newer (we use ES modules: `"type": "module"` in `package.json`).
- A terminal and `curl` (or Postman/Insomnia).
- Optional: a browser, because Apollo Server shows **Apollo Sandbox** at `/graphql` outside production.

9.2 Hello GraphQL with plain graphql-js + Express

First, the smallest possible server, so you see there is no magic. We use the official `graphql` package (the engine) and `graphql-http` (a tiny spec-compliant HTTP handler for Express).

terminal

```
mkdir hello-graphql && cd hello-graphql
npm init -y
npm pkg set type=module
npm install express graphql graphql-http
```

server.js

```
import express from "express";
import { buildSchema } from "graphql";
import { createHandler } from "graphql-http/lib/use/express";

// 1) Schema written in SDL (Schema Definition Language)
const schema = buildSchema(`
  type Query {
    hello: String!
    add(a: Int!, b: Int!): Int!
  }
`);

// 2) Resolvers: one function per field (root object = the Query resolvers)
const root = {
  hello: () => "Hello, GraphQL!",
  add: ({ a, b }) => a + b,
};

const app = express();
app.all("/graphql", createHandler({ schema, rootValue: root }));
app.listen(4200, () => console.log("http://localhost:4200/graphql"));
```

Run `node server.js`, then send a query:

terminal (tested output)

```
curl -s localhost:4200/graphql \
  -H 'content-type: application/json' \
  -d '{"query":"{ hello add(a:2,b:3) }"}'

# {"data":{"hello":"Hello, GraphQL!","add":5}}
```

What just happened?

1. `buildSchema` parsed the SDL into an executable schema object.

2. `root` supplied resolvers. For the root `Query`, graphql-js looks up `rootValue.hello` and `rootValue.add`; the args arrive as the first parameter (`{ a, b }`).
3. `createHandler` turned Express requests into GraphQL executions: parse, validate, execute, and JSON response.
4. `Int!` for `a` and `b` meant the server validated the types **before** calling your code. Try `add(a: "x", b: 1)` and read the validation error.

No server at all: calling the engine directly

GraphQL is a library first. You can run queries in-process (great for tests and scripts):

```
in-process execution (tested)
import { graphql, buildSchema } from "graphql";

const schema = buildSchema("type Query { hello: String! }");
const result = await graphql({
  schema,
  source: "{ hello }",
  rootValue: { hello: () => "hi" },
});
console.log(JSON.stringify(result)); // {"data":{"hello":"hi"}}
```

> **KEY IDEA** An HTTP server is just a convenience wrapper. The core of GraphQL is: **schema + resolvers + a query string, in; JSON out.**

10. The real project: a blog API

Now we build something realistic: **users, posts, and comments** with login, roles, pagination, search (a union), live comment notifications, and production hardening.

10.1 Architecture

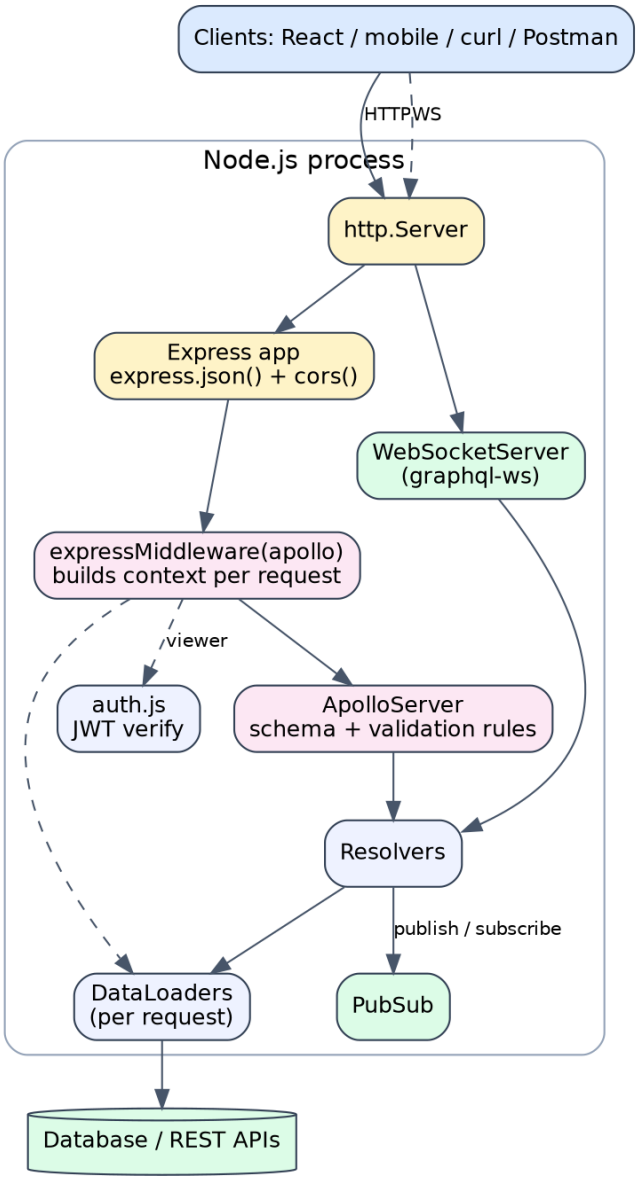


Figure 10.1: One Node.js process serves HTTP and WebSocket traffic from the same schema and resolvers.

10.2 Folder structure and installation

```
layout
blog-api/
  package.json
  src/
    data.js        in-memory data (replace with a DB)
    schema.js      the SDL (type definitions)
    auth.js        JWT helpers and requireAuth
    loaders.js     DataLoader factories (per request)
```

```
    pubsub.js      PubSub for subscriptions
    resolvers.js   the behaviour of every field
    server.js      Express + Apollo + WebSocket wiring
```

terminal

```
mkdir blog-api && cd blog-api
npm init -y
npm pkg set type=module
npm install express @apollo/server @as-integrations/express5 graphql cors \
  dataloader jsonwebtoken graphql-ws ws graphql-subscriptions \
  @graphql-tools/schema graphql-depth-limit
```

Package	Why we need it
`express`	The HTTP framework (v5)
`@apollo/server`	GraphQL server with great errors, plugins, and the Sandbox UI
`@as-integrations/express5`	Provides `expressMiddleware` to mount Apollo on Express 5 (use `@as-integrations/express4` for Express 4)
`graphql`	The reference engine (peer dependency)
`dataloader`	Batching and caching to fix N+1
`jsonwebtoken`	Sign and verify JWT tokens
`graphql-ws` + `ws`	WebSocket subscriptions protocol and server
`graphql-subscriptions`	In-memory PubSub
`@graphql-tools/schema`	`makeExecutableSchema`, so HTTP and WebSocket share one schema
`graphql-depth-limit`	Validation rule that rejects overly deep queries
`cors`	Allow browser apps on other origins to call the API

> **NOTE** Library APIs change between major versions. If an import fails after a future upgrade, check the package's current docs. The code here was verified with: @apollo/server 5.5, @as-integrations/express5 1.1, express 5.3, graphql 16.14, graphql-ws 6.4, dataloader 2.2.

11. Walking through the code

11.1 data.js: our fake database

A plain module with arrays. Everything else would look the same with Postgres, MongoDB, or Prisma; only the loaders and resolvers would call the database instead.

data.js

```
// In-memory "database" — swap for Postgres/Mongo later without touching the schema.
export const users = [
  { id: "1", name: "Asha", email: "asha@example.com", role: "ADMIN", password: "admin123" },
  { id: "2", name: "Ravi", email: "ravi@example.com", role: "USER", password: "user123" },
];

export const posts = [
  { id: "1", title: "GraphQL 101", body: "Types, queries, mutations.", authorId: "1", published:
true, createdAt: "2026-01-01" },
  { id: "2", title: "Express Tips", body: "Middleware everywhere.", authorId: "1", published: true,
createdAt: "2026-01-02" },
  { id: "3", title: "Draft Idea", body: "Not ready yet.", authorId: "2", published: false, createdAt:
"2026-01-03" },
];

export const comments = [
  { id: "1", text: "Great post!", postId: "1", authorId: "2" },
  { id: "2", text: "Thanks for sharing", postId: "1", authorId: "1" },
  { id: "3", text: "Waiting for more", postId: "2", authorId: "2" },
];

let nextId = 100;
export const newId = () => String(nextId++);
```

11.2 schema.js: the contract

The entire API surface lives here. Notice **enum**, **input types**, a **union**, **cursor pagination types**, and the three root types.

schema.js

```
export const typeDefs = /* GraphQL */ `
  enum Role { USER ADMIN }

  type User {
    id: ID!
    name: String!
    email: String!
    role: Role!
    posts: [Post!]!
  }

  type Post {
    id: ID!
    title: String!
    body: String!
    published: Boolean!
    createdAt: String!
    author: User!
    comments: [Comment!]!
  }

  type Comment {
    id: ID!
    text: String!
    author: User!
    post: Post!
```

```
  }

  # Relay-style cursor pagination
  type PostEdge { cursor: String!, node: Post! }
  type PageInfo { hasNextPage: Boolean!, endCursor: String }
  type PostConnection { edges: [PostEdge!]!, pageInfo: PageInfo!, totalCount: Int! }

  union SearchResult = User | Post | Comment

  input CreatePostInput {
    title: String!
    body: String!
    published: Boolean = false
  }

  input UpdatePostInput {
    title: String
    body: String
    published: Boolean
  }

  type AuthPayload { token: String!, user: User! }

  type Query {
    me: User
    user(id: ID!): User
    users: [User!]!
    post(id: ID!): Post
    posts(first: Int = 10, after: String): PostConnection!
    search(term: String!): [SearchResult!]!
  }

  type Mutation {
    login(email: String!, password: String!): AuthPayload!
    createPost(input: CreatePostInput!): Post!
    updatePost(id: ID!, input: UpdatePostInput!): Post!
    deletePost(id: ID!): Boolean!
    addComment(postId: ID!, text: String!): Comment!
  }

  type Subscription {
    commentAdded(postId: ID!): Comment!
  }
`;
```

> **KEY IDEA** A client can learn the whole API from this one file. That is the power of a typed schema: it is documentation, validation rules, and the contract between frontend and backend at the same time.

11.3 auth.js: JWT and the viewer

`getViewer` turns an `Authorization: Bearer ...` header into `{ id, role }` or `null`. An invalid token simply means "anonymous", and the **resolvers decide** what anonymous users may do. `requireAuth` throws a `GraphQLError` with a machine-readable `code` and an HTTP status override (`401`).

auth.js

```
import jwt from "jsonwebtoken";
import { GraphQLError } from "graphql";

const SECRET = process.env.JWT_SECRET || "dev-secret-change-me";
```

```
export const signToken = (user) =>
  jwt.sign({ sub: user.id, role: user.role }, SECRET, { expiresIn: "1d" });

// Reads "Authorization: Bearer <token>" and returns { id, role } or null
export function getViewer(authHeader = "") {
  const token = authHeader.replace("Bearer ", "");
  if (!token) return null;
  try {
    const payload = jwt.verify(token, SECRET);
    return { id: payload.sub, role: payload.role };
  } catch {
    return null; // invalid/expired token => treated as anonymous
  }
}

export function requireAuth(ctx) {
  if (!ctx.viewer) {
    throw new GraphQLError("You must be logged in", {
      extensions: { code: "UNAUTHENTICATED", http: { status: 401 } },
    });
  }
  return ctx.viewer;
}
```

PITFALL The demo stores passwords in plain text and uses a default JWT secret for brevity. In a real app, hash passwords with bcrypt or argon2, load `JWT_SECRET` from environment variables, and never commit secrets.

11.4 loaders.js: killing the N+1 problem

`createLoaders()` returns a **new** set of DataLoaders each time it is called. We call it once per request from the context function. Notice the `console.log` lines: they let you **see** the batching later.

loaders.js

```
import DataLoader from "dataloader";
import { users, posts, comments } from "./data.js";

// A NEW set of loaders is created for every request (per-request cache!)
export function createLoaders() {
  return {
    userById: new DataLoader(async (ids) => {
      console.log("  [batch] users", ids);
      const map = new Map(users.map((u) => [u.id, u]));
      return ids.map((id) => map.get(id) ?? null); // same order as ids!
    }),
    postById: new DataLoader(async (ids) => {
      const map = new Map(posts.map((p) => [p.id, p]));
      return ids.map((id) => map.get(id) ?? null);
    }),
    commentsByPostId: new DataLoader(async (postIds) => {
      console.log("  [batch] comments for posts", postIds);
      return postIds.map((pid) => comments.filter((c) => c.postId === pid));
    }),
    postsByAuthorId: new DataLoader(async (authorIds) =>
      authorIds.map((aid) => posts.filter((p) => p.authorId === aid))
    ),
  };
}
```

11.5 pubsub.js

pubsub.js

```
import { PubSub } from "graphql-subscriptions";
```

```
export const pubsub = new PubSub();
export const EVENTS = { COMMENT_ADDED: "COMMENT_ADDED" };
```

11.6 resolvers.js: the behaviour

Read this file top to bottom. Key patterns to notice:

- **Query resolvers** use `loaders` for single objects and filter visible posts by the viewer (drafts are visible only to the author and admins).
- **Cursor pagination** in `posts`: cursors are base64 post ids; `hasNextPage` and `endCursor` go into `pageInfo`.
- **Mutations** call `requireAuth(ctx)` first; `updatePost` and `deletePost` also check ownership or the `ADMIN` role (**authorization**).
- **Field resolvers** (`Post.author`, `Post.comments`, `User.posts`, ...) express relationships and all go through loaders.
- **Field-level security:** `User.email` returns the real address only to the owner or an admin.
- **`SearchResult.__resolveType`** tells GraphQL which concrete type each union member is.
- **Subscription** resolver is an async iterator that filters events by `postId`.

resolvers.js

```
import { GraphQLError } from "graphql";
import { users, posts, comments, newId } from "./data.js";
import { signToken, requireAuth } from "./auth.js";
import { pubsub, EVENTS } from "./pubsub.js";

const canSee = (post, viewer) =>
  post.published || viewer?.role === "ADMIN" || viewer?.id === post.authorId;

export const resolvers = {
  Query: {
    me: (_, __, { viewer, loaders }) => (viewer ? loaders.userById.load(viewer.id) : null),
    user: (_, { id }, { loaders }) => loaders.userById.load(id),
    users: () => users,
    post: async (_, { id }, { loaders, viewer }) => {
      const post = await loaders.postById.load(id);
      return post && canSee(post, viewer) ? post : null;
    },
    posts: (_, { first, after }, { viewer }) => {
      const visible = posts.filter((p) => canSee(p, viewer));
      const start = after ? visible.findIndex((p) => p.id === Buffer.from(after,
"base64").toString()) + 1 : 0;
      const slice = visible.slice(start, start + first);
      const edges = slice.map((p) => ({ cursor: Buffer.from(p.id).toString("base64"), node: p }));
      return {
        edges,
        totalCount: visible.length,
        pageInfo: {
          hasNextPage: start + first < visible.length,
          endCursor: edges.length ? edges[edges.length - 1].cursor : null,
        },
      };
    },
    search: (_, { term }) => {
      const t = term.toLowerCase();
      return [
        ...users.filter((u) => u.name.toLowerCase().includes(t)),
        ...posts.filter((p) => p.published && p.title.toLowerCase().includes(t)),
        ...comments.filter((c) => c.text.toLowerCase().includes(t)),
      ];
    },
  },
```

```
  Mutation: {
    login: (_, { email, password }) => {
      const user = users.find((u) => u.email === email && u.password === password);
      if (!user) {
        throw new GraphQLError("Invalid credentials", { extensions: { code: "BAD_CREDENTIALS" } });
      }
      return { token: signToken(user), user };
    },
    createPost: (_, { input }, ctx) => {
      const viewer = requireAuth(ctx);
      const post = { id: newId(), ...input, authorId: viewer.id, createdAt: new Date().toISOString()
};
      posts.push(post);
      return post;
    },
    updatePost: (_, { id, input }, ctx) => {
      const viewer = requireAuth(ctx);
      const post = posts.find((p) => p.id === id);
      if (!post) throw new GraphQLError("Post not found", { extensions: { code: "NOT_FOUND" } });
      if (post.authorId !== viewer.id && viewer.role !== "ADMIN") {
        throw new GraphQLError("Not allowed", { extensions: { code: "FORBIDDEN" } });
      }
      Object.assign(post, Object.fromEntries(Object.entries(input).filter(([, v]) => v !==
undefined)));
      return post;
    },
    deletePost: (_, { id }, ctx) => {
      const viewer = requireAuth(ctx);
      const i = posts.findIndex((p) => p.id === id);
      if (i === -1) return false;
      if (posts[i].authorId !== viewer.id && viewer.role !== "ADMIN") {
        throw new GraphQLError("Not allowed", { extensions: { code: "FORBIDDEN" } });
      }
      posts.splice(i, 1);
      return true;
    },
    addComment: async (_, { postId, text }, ctx) => {
      const viewer = requireAuth(ctx);
      const post = await ctx.loaders.postById.load(postId);
      if (!post) throw new GraphQLError("Post not found", { extensions: { code: "NOT_FOUND" } });
      const comment = { id: newId(), text, postId, authorId: viewer.id };
      comments.push(comment);
      pubsub.publish(EVENTS.COMMENT_ADDED, { commentAdded: comment });
      return comment;
    },
  },

  Subscription: {
    commentAdded: {
      subscribe: (_, { postId }) => ({
        // async iterator; filter inside so each client only gets its post
        async *[Symbol.asyncIterator]() {
          for await (const evt of pubsub.asyncIterableIterator(EVENTS.COMMENT_ADDED)) {
            if (evt.commentAdded.postId === postId) yield evt;
          }
        },
      }),
    },
  },

  // Field resolvers for relationships — each runs once per parent object
  User: {
    posts: (user, _, { loaders }) => loaders.postsByAuthorId.load(user.id),
    // only the owner or admin may read an email
    email: (user, _, { viewer }) =>
      viewer && (viewer.id === user.id || viewer.role === "ADMIN") ? user.email : "hidden",
  },
  Post: {
    author: (post, _, { loaders }) => loaders.userById.load(post.authorId),
    comments: (post, _, { loaders }) => loaders.commentsByPostId.load(post.id),
  },
  Comment: {
    author: (c, _, { loaders }) => loaders.userById.load(c.authorId),
    post: (c, _, { loaders }) => loaders.postById.load(c.postId),
  },
```

```
  // Unions/interfaces need to know the concrete type
  SearchResult: {
    __resolveType(obj) {
      if (obj.email) return "User";
      if (obj.title) return "Post";
      return "Comment";
    },
  },
};
```

11.7 server.js: wiring Express, Apollo, and WebSocket together

server.js

```
import http from "node:http";
import express from "express";
import cors from "cors";
import { ApolloServer } from "@apollo/server";
import { expressMiddleware } from "@as-integrations/express5";
import { ApolloServerPluginDrainHttpServer } from "@apollo/server/plugin/drainHttpServer";
import { makeExecutableSchema } from "@graphql-tools/schema";
import { WebSocketServer } from "ws";
import { useServer } from "graphql-ws/use/ws";
import depthLimit from "graphql-depth-limit";

import { typeDefs } from "./schema.js";
import { resolvers } from "./resolvers.js";
import { createLoaders } from "./loaders.js";
import { getViewer } from "./auth.js";

const schema = makeExecutableSchema({ typeDefs, resolvers });

const app = express();
const httpServer = http.createServer(app);

// WebSocket endpoint for subscriptions (same port, path /graphql)
const wsServer = new WebSocketServer({ server: httpServer, path: "/graphql" });
const wsCleanup = useServer(
  {
    schema,
    context: (ctx) => ({
      viewer: getViewer(ctx.connectionParams?.authorization),
      loaders: createLoaders(),
    }),
  },
  wsServer
);

const apollo = new ApolloServer({
  schema,
  validationRules: [depthLimit(6)], // reject absurdly deep queries
  introspection: process.env.NODE_ENV !== "production",
  plugins: [
    ApolloServerPluginDrainHttpServer({ httpServer }),
    { async serverWillStart() { return { async drainServer() { await wsCleanup.dispose(); } }; } },
  ],
});
await apollo.start();

app.use(
  "/graphql",
  cors(),
  express.json(),
  expressMiddleware(apollo, {
    // runs for EVERY request -> builds the per-request context
    context: async ({ req }) => ({
      viewer: getViewer(req.headers.authorization),
      loaders: createLoaders(),
    }),
  })
```

```
);

app.get("/health", (_, res) => res.json({ ok: true }));

const PORT = process.env.PORT || 4000;
httpServer.listen(PORT, () => console.log(`GraphQL ready at http://localhost:${PORT}/graphql`));
```

Line-by-line ideas

- `makeExecutableSchema` builds **one schema object** shared by HTTP and WebSocket.
- We create `http.createServer(app)` ourselves (instead of `app.listen`) so the **WebSocketServer can attach to the same port**.
- `useServer` (from `graphql-ws`) handles subscription connections. Its own `context` function authenticates using `connectionParams`, because WebSockets do not carry per-message HTTP headers.
- `validationRules: [depthLimit(6)]` plugs into the **validate** phase, so deep queries die before any resolver runs.
- `introspection` is switched off when `NODE_ENV=production`.
- `ApolloServerPluginDrainHttpServer` plus the dispose plugin give graceful shutdown.
- `app.use("/graphql", cors(), express.json(), expressMiddleware(...))`: the usual Express middleware chain, with Apollo as the last link.
- The `context` function runs for **every request** and builds `{ viewer, loaders }`, which is exactly what resolvers receive as their third argument.

> **NOTE** `express.json()` must come **before** `expressMiddleware`, or Apollo will not see the request body.

11.8 Run it

terminal

```
npm start
# GraphQL ready at http://localhost:4000/graphql


# development with auto-restart on file changes:
npm run dev


# production-like mode (hides Sandbox, disables introspection):
NODE_ENV=production JWT_SECRET=a-long-random-secret npm start
```

Open `http://localhost:4000/graphql` in a browser to use Apollo Sandbox: it reads your schema through introspection and gives you autocomplete, docs, and a request history.

12. Trying every feature

Each snippet below was run against the server. To keep commands short, define this helper in your shell once (it sends a JSON body and optionally a token):

bash helper

```
gql() {  # usage: gql 'query text' [TOKEN]
  curl -s localhost:4000/graphql \
    -H 'content-type: application/json' \
    ${2:+-H "authorization: Bearer $2"} \
    -d "$(node -e 'console.log(JSON.stringify({query: process.argv[1]}))' "$1")"
  echo
}
```

12.1 A nested query: one request, many relations

request

```
gql '{
  posts(first: 2) {
    totalCount
    pageInfo { hasNextPage endCursor }
    edges { node { title author { name } comments { text author { name } } } }
  }
}'
```

response (tested; formatted for reading)

```
{ "data": { "posts": {
    "totalCount": 2,
    "pageInfo": { "hasNextPage": false, "endCursor": "Mg==" },
    "edges": [
      { "node": { "title": "GraphQL 101", "author": { "name": "Asha" },
          "comments": [
            { "text": "Great post!",        "author": { "name": "Ravi" } },
            { "text": "Thanks for sharing", "author": { "name": "Asha" } } ] } },
      { "node": { "title": "Express Tips", "author": { "name": "Asha" },
          "comments": [ { "text": "Waiting for more", "author": { "name": "Ravi" } } ] } }
    ] } } }
```

Seeing DataLoader at work

In the server terminal this single request printed:

server log (tested)

```
  [batch] users [ '1' ]
  [batch] comments for posts [ '1', '2' ]
  [batch] users [ '2' ]
```

The two posts share the same author, so the **first batch asked for just user 1**. Both posts' comments were fetched in **one batch** for posts 1 and 2. The comment authors needed user 2 (user 1 was already cached), giving a third batch. Without DataLoader the same query would have made about **seven separate lookups** (2 post authors, 2 comment lists, 3 comment authors). With it: **three batched calls**, and the gap grows quickly with bigger lists.

12.2 Login and use the token

login (tested)

```
gql 'mutation { login(email: "ravi@example.com", password: "user123") { token user { name } } }'
# {"data":{"login":{"token":"eyJhbGciOi...","user":{"name":"Ravi"}}}}
```

```
TOKEN=eyJhbGciOi...   # paste the token from the response
```

12.3 Authentication errors

no token

```
gql 'mutation { createPost(input: { title: "x", body: "y" }) { id } }'
```

response (trimmed)

```
{
  "errors": [{
    "message": "You must be logged in",
    "path": ["createPost"],
    "extensions": { "code": "UNAUTHENTICATED", ... }
  }],
  "data": null
}
# HTTP status: 401 (verified)
```

12.4 Create a post with variables

tested

```
curl -s localhost:4000/graphql \
  -H 'content-type: application/json' -H "authorization: Bearer $TOKEN" \
  -d '{
    "query": "mutation($i: CreatePostInput!) { createPost(input: $i) { id title published author {
name } } }",
    "variables": { "i": { "title": "Ravi post", "body": "hello" } }
  }'

# {"data":{"createPost":{"id":"100","title":"Ravi post","published":false,"author":{"name":"Ravi"}}}}
```

`published` came back `false` because the **input default** (`published: Boolean = false`) was applied by GraphQL itself.

12.5 Field-level security and authorization

tested

```
gql '{ me { name email role } }' "$TOKEN"
# {"data":{"me":{"name":"Ravi","email":"ravi@example.com","role":"USER"}}}

gql 'mutation { deletePost(id: "1") }' "$TOKEN"
# error: { "message": "Not allowed", "extensions": { "code": "FORBIDDEN" } }   (Ravi is not the
author)
```

12.6 Union search

tested

```
gql '{ search(term: "great") { __typename ... on Comment { text } ... on Post { title } ... on User {
name } } }'
# {"data":{"search":[{"__typename":"Comment","text":"Great post!"}]}}
```

12.7 Validation and depth-limit errors

tested

```
gql '{ post(id: "1") { nope } }'
# "Cannot query field \"nope\" on type \"Post\"."   code: GRAPHQL_VALIDATION_FAILED  (HTTP 400)
```

```
# A query nested 9 levels deep:
# "'' exceeds maximum operation depth of 6"          code: GRAPHQL_VALIDATION_FAILED
```

Both failures happened in the **validation phase**, so none of your resolvers ran.

12.8 Subscriptions end-to-end

Create a tiny Node client with `graphql-ws`. It subscribes to comments on post 1 and prints events. Then trigger `addComment` mutations.

sub-client.mjs

```
// sub-client.mjs   (run: node sub-client.mjs)
import { createClient } from "graphql-ws";
import WebSocket from "ws";

const client = createClient({
  url: "ws://localhost:4000/graphql",
  webSocketImpl: WebSocket,
  connectionParams: { authorization: "" }, // put "Bearer <token>" here if needed
});

client.subscribe(
  { query: 'subscription { commentAdded(postId: "1") { id text author { name } } }' },
  {
    next: (data) => console.log("EVENT", JSON.stringify(data)),
    error: console.error,
    complete: () => console.log("done"),
  }
);
```

In another terminal, log in as Asha (`admin123`) and add comments to posts 2 and then 1:

trigger an event

```
gql 'mutation { addComment(postId: "1", text: "live on 1") { id } }' "$ASHA_TOKEN"
```

client output (tested: only the post-1 comment arrived; the post-2 comment was filtered out)

```
EVENT {"data":{"commentAdded":{"id":"102","text":"live on 1","author":{"name":"Asha"}}}}
```

> **TIP** In Apollo Sandbox you can also run subscriptions: set the connection settings to the `graphql-ws` protocol and point it at `ws://localhost:4000/graphql`.

13. Testing, errors and production hardening

13.1 Testing without HTTP

Apollo Server can run an operation directly with `executeOperation`. You pass the context yourself, so tests are fast and need no ports:

test.js (works with node:test, Jest, Vitest)

```
import { ApolloServer } from "@apollo/server";
import { makeExecutableSchema } from "@graphql-tools/schema";
import { typeDefs } from "./schema.js";
import { resolvers } from "./resolvers.js";
import { createLoaders } from "./loaders.js";

const server = new ApolloServer({ schema: makeExecutableSchema({ typeDefs, resolvers }) });

const res = await server.executeOperation(
  { query: "query($id: ID!){ post(id:$id){ title author{ name } } }", variables: { id: "1" } },
  { contextValue: { viewer: null, loaders: createLoaders() } }
);

// res.body.kind === "single"
console.log(res.body.singleResult.data);
// { post: { title: 'GraphQL 101', author: { name: 'Asha' } } }   (tested)
await server.stop();
```

In a real test you would assert with your test framework, for example `assert.equal(res.body.singleResult.errors, undefined)`. For full-stack tests, `supertest` can POST to your Express app.

13.2 Throwing good errors

error patterns

```
import { GraphQLError } from "graphql";

throw new GraphQLError("Post not found", {
  extensions: { code: "NOT_FOUND", http: { status: 404 } },
});

// Input validation:
throw new GraphQLError("Title must be at least 3 characters", {
  extensions: { code: "BAD_USER_INPUT", argumentName: "title" },
});
```

13.3 Hide internals in production

In development Apollo includes a `stacktrace` in each error (you saw it in our test outputs). In production, turn it off and mask unexpected errors:

server.js additions

```
const apollo = new ApolloServer({
  schema,
  includeStacktraceInErrorResponses: process.env.NODE_ENV !== "production",
  formatError: (formatted, error) => {
    // keep deliberate errors, mask unexpected crashes
    if (formatted.extensions?.code === "INTERNAL_SERVER_ERROR") {
      console.error(error); // log the real cause
      return { message: "Something went wrong", extensions: { code: "INTERNAL_SERVER_ERROR" } };
    }
    return formatted;
```

```
  },
  // ...rest of options
});
```

13.4 Production checklist

Area	Action
Secrets	Load `JWT_SECRET` and DB URLs from environment variables
Passwords	Hash with bcrypt/argon2; never store plain text
Introspection and Sandbox	Disabled in production (`NODE_ENV=production`), or protected
Limits	Depth limit (done), add cost analysis, cap `first` (for example max 100), request size limits, timeouts
Rate limiting	`express-rate-limit` before the GraphQL middleware, or at your gateway
CORS	Restrict `cors({ origin: [...] })` to your real frontends
Logging and tracing	Apollo plugins, OpenTelemetry, log operation names and durations
Batching and caching	DataLoader per request; Redis or response caching for hot data
Subscriptions at scale	Replace in-memory PubSub with Redis/Kafka-backed PubSub
Persisted queries	Allow-list known operations for public apps
Graceful shutdown	Drain plugin (done) and handle `SIGTERM`

13.5 Swapping in a real database

Only two places change: the **batch functions in `loaders.js`** and the **write operations in `resolvers.js`**. For example with Prisma:

loaders.js with a database (pattern)

```
userById: new DataLoader(async (ids) => {
  const rows = await prisma.user.findMany({ where: { id: { in: ids.map(Number) } } });
  const map = new Map(rows.map((u) => [String(u.id), u]));
  return ids.map((id) => map.get(id) ?? null);
}),
```

14. Calling your API from a client

14.1 Plain fetch (browser or Node 18+)

client.js

```
async function gql(query, variables, token) {
  const res = await fetch("http://localhost:4000/graphql", {
    method: "POST",
    headers: {
      "content-type": "application/json",
      ...(token ? { authorization: "Bearer " + token } : {}),
    },
    body: JSON.stringify({ query, variables }),
  });
  const json = await res.json();
  if (json.errors) throw new Error(json.errors.map((e) => e.message).join("; "));
  return json.data;
}

const data = await gql(
  "query($id: ID!) { post(id: $id) { title author { name } } }",
  { id: "1" }
);
console.log(data.post.title);
```

14.2 Apollo Client (React) in one glance

React (pattern)

```
import { ApolloClient, InMemoryCache, gql, useQuery } from "@apollo/client";

const client = new ApolloClient({ uri: "http://localhost:4000/graphql", cache: new InMemoryCache()
});

const POSTS = gql`
  query { posts(first: 5) { edges { node { id title author { name } } } } }
`;

function Posts() {
  const { data, loading, error } = useQuery(POSTS);
  if (loading) return <p>Loading...</p>;
  if (error) return <p>{error.message}</p>;
  return <ul>{data.posts.edges.map(({ node }) => <li key={node.id}>{node.title} by
{node.author.name}</li>)}</ul>;
}
```

Apollo Client's `InMemoryCache` is the **normalised cache** described in chapter 7: because every object has `id` and `__typename`, a mutation that returns an updated `Post` automatically refreshes every component displaying it.

14.3 Type-safe clients with code generation

GraphQL Code Generator reads your schema and your client queries and generates TypeScript types and typed hooks, so a typo in a field name becomes a compile error instead of a runtime error. Typical flow: write `.graphql` documents, run `graphql-codegen`, import generated hooks.

15. Becoming a hero: practice and next steps

15.1 Build-it-yourself challenges

Learning sticks when you extend the project. Try these in order; each one reinforces a concept from the guide.

1. **Add `Tag`s:** a `Tag` type with a many-to-many relation to `Post` (`Post.tags`, `Query.postsByTag`). Add a `tagsByPostId` DataLoader.
2. **Add `likePost` mutation** and a `likeCount` field. Then a `postLiked` subscription.
3. **Interfaces:** introduce `interface Node { id: ID! }`, make `User`, `Post`, and `Comment` implement it, and add `node(id: ID!): Node` using `__resolveType`.
4. **Typed errors:** change `createPost` to return `union CreatePostResult = Post | ValidationError` and reject titles shorter than 3 characters.
5. **Custom scalar:** replace `createdAt: String!` with a `DateTime` scalar (try `graphql-scalars`).
6. **Real database:** replace `data.js` with SQLite/Postgres via Prisma or Drizzle, keeping DataLoaders.
7. **Cost limiting:** add a query cost analysis rule and cap `first` at 50.
8. **TypeScript:** convert the project and add GraphQL Code Generator for resolver types.
9. **Redis PubSub:** run two server instances and make subscriptions work across both.
10. **Federation:** split `Comment` into its own subgraph behind an Apollo Router.

15.2 Where to go next

- The official site **graphql.org/learn** for the spec-level explanations.
- Apollo's documentation for Server, Client, and Federation.
- The GraphQL specification itself, once you want exact execution semantics.
- Relay's connection spec, when you want to understand cursor pagination formally.
- Study a production schema (for example GitHub's public GraphQL API) in a tool like Apollo Sandbox to see mature design patterns.

16. Cheat sheet and glossary

16.1 SDL quick reference

```
SDL
scalar DateTime
enum Role { USER ADMIN }
interface Node { id: ID! }
type User implements Node { id: ID!  name: String!  posts: [Post!]! }
union SearchResult = User | Post
input NewPost { title: String!  body: String }
type Query        { user(id: ID!): User  search(term: String!): [SearchResult!]! }
type Mutation     { createPost(input: NewPost!): Post! }
type Subscription { postCreated: Post! }
```

16.2 Query quick reference

```
query language
query Name($v: Type = default) {        # operation + variables
  alias: field(arg: $v) @include(if: $b) {
    ...FragName                         # fragment spread
    ... on Type { field }               # inline fragment
    __typename                          # concrete type name
  }
}
fragment FragName on Type { field }
mutation { doThing(input: {...}) { id } }
subscription { somethingHappened { id } }
```

16.3 Glossary

Term	Definition
AST	Abstract syntax tree: the parsed form of a query
Connection	Relay pattern for pagination: edges, nodes, cursors, pageInfo
Context	Per-request object passed to every resolver
DataLoader	Utility that batches and caches loads to prevent N+1
Directive	Annotation such as `@include`, `@skip`, `@deprecated`
Federation	Composing multiple GraphQL services (subgraphs) into one supergraph
Fragment	Reusable group of fields on a type
Introspection	Querying a schema about itself (`__schema`, `__type`)
Mutation	Operation that writes data
N+1 problem	1 query for a list plus N queries for related items
Persisted query	Pre-registered query referenced by hash or id
PubSub	Publish/subscribe channel used for subscriptions
Resolver	Function that returns the value for a field
SDL	Schema Definition Language
Subscription	Long-lived operation that streams results
Union	A field that can be one of several object types

You now have the full path: the ideas, the type system, the query language, execution, and a complete tested Express.js project. The best next step is the first challenge in section 15.1: extend the blog with tags and watch DataLoader batch the new relation.